

Necessary Library Functions

```
img = cv2.imread('lena.jpg', cv2.IMREAD_GRAYSCALE)

img_bordered = cv2.copyMakeBorder(img, 2, 2, 2, 2, cv2.BORDER_CONSTANT)

norm = np.round(cv2.normalize(img_conv, None, 0, 255, cv2.NORM_MINMAX)).astype(np.uint8)

cv2.imshow('Original Grayscale Image', img)

cv2.waitKey(0)

cv2.destroyAllWindows()

result = cv2.filter2D(image, ddepth=cv2.CV_32F, kernel=kernel)

norm_cropped = norm[pad_h:h+pad_h, pad_w:w+pad_w]

img_hsv = cv2.cvtColor(img_color, cv2.COLOR_BGR2HSV)

h, s, v = cv2.split(img_hsv)

merged_hsv = cv2.merge([h, s, v])

img_merged_bgr = cv2.cvtColor(merged_hsv, cv2.COLOR_HSV2BGR)

gradient_magnitude = cv2.magnitude(gx.astype(np.float32), gy.astype(np.float32))

histogram = cv2.calcHist(images, channels, mask, histSize, ranges[, hist[, accumulate]])

plt.figure(figsize=(12,16))
plt.subplot(3,2,1)
plt.imshow(img, cmap='gray')
plt.title("Original Image")
plt.axis("off")

plt.subplot(3,2,6)
plt.plot(cdf_out, color='red')
plt.title("Equalized CDF")

plt.tight_layout()
plt.show()

eroded = cv2.erode(image, kernel, iterations=1)

contours, __ = cv2.findContours(img,cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
cnt = max(contours, key=cv2.contourArea)

(x, y), (MA, ma), angle = cv2.fitEllipse(cnt)

row_headers = [f'Test {i + 1}' for i in range(4)]
col_headers = [f'GT {i + 1}' for i in range(3)]
distances_matrix = np.array(distances_matrix)
print(tabulate(distances_matrix[0:7,0:7], headers=col_headers, showindex=row_headers,
tablefmt='grid'))
```

```
ft = np.fft.fft2(img)
```

```
ft_shift = np.fft.fftshift(ft)
```

```
fft_ang = np.angle(ft_shift)
```

```
fft_magnitude_spectrum_ac = np.abs(ft_shift)
```

```
fft_final_result = np.multiply(magnitude_spectrum_ac, np.exp(1j*ang))
```

```
fft_img_back = np.real(np.fft.ifft2(np.fft.ifftshift(final_result)))
```